

LANGUAGE REFERENCE // JS · TS · NODE

JavaScript, TypeScript & Node.js

A focused reference spanning ES6 (ES2015) syntax, core JavaScript concepts, classes in depth, TypeScript's type system, and the Node.js runtime.

Covers

ES6, JS core, TS, Node.js

Format

Syntax + short examples

Prepared

September 20, 2026

CONTENTS

01	let & const	Block scoping
02	Arrow functions	Shorter syntax, lexical this
03	Template literals	String interpolation
04	Default & rest parameters	Flexible function signatures
05	Spread operator	Expanding arrays & objects
06	Destructuring	Unpacking values
07	Enhanced object literals	Shorthand & computed keys
08	for...of & iterables	Iteration protocol
09	Classes	Syntax over prototypes
10	Modules	import / export
11	Promises	Async without callback hell
12	Map & Set	New collection types
13	Symbols	Unique property keys
14	Generators	Pausable functions
15	Classes in depth	Fields, statics, mixins, patterns
16	JavaScript core	Beyond ES6: JS every day
17	TypeScript	Types, interfaces, generics
18	Node.js	The server-side runtime
19	Quick reference	One-page cheat sheet

let & const

ES6 introduced two new ways to declare variables that are scoped to the nearest block `{ }`, replacing the function-scoped and hoisting quirks of `var`.

let reassignable, block-scoped

```
let count = 1;
count = 2;           // OK: let can be reassigned

if (true) {
  let count = 99;    // a *different* binding inside the block
}
console.log(count);  // 2 – outer count is untouched
```

const single assignment, block-scoped

```
const PI = 3.14159;
PI = 3;              // TypeError: Assignment to constant variable

const user = { name: "Ana" };
user.name = "Beatriz"; // OK – the object's *contents* are still mutable
user = {};             // TypeError – only the binding is locked
```

BEFORE – VAR

```
for (var i = 0; i < 3; i++) {
  setTimeout(() => console.log(i), 0);
}
// logs 3, 3, 3
```

ES6 – LET

```
for (let i = 0; i < 3; i++) {
  setTimeout(() => console.log(i), 0);
}
// logs 0, 1, 2
```

`var` is function-scoped and hoisted with an `undefined` value; every loop iteration above shares one `i`. `let` creates a fresh binding per iteration, which is why closures over it behave as expected.

Rule of thumb: default to `const`. Reach for `let` only when a variable genuinely needs reassignment. Avoid `var` in new ES6+ code.

Arrow functions

A shorter function syntax that also changes how `this` is resolved — arrows don't bind their own `this`, arguments, or `super`; they inherit them from the enclosing scope.

Syntax forms

```
const add = (a, b) => a + b;           // implicit return, single expression
const square = x => x * x;             // one parameter: parens optional
const greet = () => {                 // block body needs explicit return
  return "Hello";
};
const makeUser = (name) => ({ name }); // wrap an object literal in parens
```

Lexical this

BEFORE — FUNCTION

```
function Timer() {
  this.seconds = 0;
  setInterval(function () {
    this.seconds++; // 'this' is
wrong here
  }, 1000);
}
```

ES6 — ARROW

```
function Timer() {
  this.seconds = 0;
  setInterval(() => {
    this.seconds++; // 'this' is the
Timer instance
  }, 1000);
}
```

Don't use arrows for: object methods that need their own `this`, or constructor functions — arrow functions cannot be used with `new`.

Template literals

Backtick-delimited strings that support embedded expressions and multi-line text without concatenation.

Interpolation

```
const name = "Ana";
const age = 29;

// before: string concatenation
const oldMsg = "Hi " + name + ", you are " + age + " years old.";

// ES6: template literal
const msg = `Hi ${name}, you are ${age} years old.`;
const total = `Total: ${19.99 * 3}.toFixed(2)}`; // any expression works
```

Multi-line strings

```
const html = `
  <ul>
    <li>${name}</li>
  </ul>
`;
```

Tagged templates

```
function highlight(strings, ...values) {
  return strings.reduce((out, str, i) =>
    `${out}${str}${values[i] ? `**${values[i]}**` : ""}`, "");
}
highlight`Hello ${name}, you scored ${95}.`;
// "Hello **Ana**, you scored **95**."
```

The function receives the literal string chunks as an array, and the interpolated values separately — the basis for libraries like styled-components.

Default & rest parameters

Two additions that remove the need for manual argument checks and the array-like `arguments` object.

Default parameters

```
function greet(name = "friend", greeting = "Hello") {  
  return `${greeting}, ${name}!`;  
}  
greet();           // "Hello, friend!"  
greet("Ana");      // "Hello, Ana!"  
greet("Ana", "Hi"); // "Hi, Ana!"  
  
// defaults can reference earlier parameters  
function box(width, height = width) { return width * height; }
```

Rest parameters

```
function sum(...numbers) {           // collects remaining args into a real array  
  return numbers.reduce((a, b) => a + b, 0);  
}  
sum(1, 2, 3, 4);    // 10  
  
function log(level, ...messages) {   // combine fixed + rest params  
  console.log(level, messages.join(" "));  
}
```

Unlike the old `arguments` object, a rest parameter is a genuine `Array` — `map`, `filter` and `reduce` work directly on it.

Spread operator

The same `...` syntax as rest parameters, used in the opposite direction: expanding an iterable or object into individual elements.

Arrays

```
const nums = [1, 2, 3];
const more = [...nums, 4, 5];           // [1, 2, 3, 4, 5]
const copy = [...nums];                 // shallow copy
const merged = [...nums, ...[9, 8]];    // [1, 2, 3, 9, 8]

Math.max(...nums);                     // spread into function arguments
```

Objects

```
const base = { role: "user", active: true };
const admin = { ...base, role: "admin" }; // { role: "admin", active: true }
const clone = { ...base };                // shallow copy
```

Object spread copies own enumerable properties one level deep; nested objects are still shared by reference.

Destructuring

Unpack values from arrays or properties from objects into distinct variables in a single statement.

Arrays

```
const coords = [10, 20, 30];  
const [x, y] = coords;           // x = 10, y = 20  
const [, , z] = coords;         // skip elements: z = 30  
const [first, ...rest] = coords; // rest = [20, 30]  
  
let a = 1, b = 2;  
[a, b] = [b, a];                // swap without a temp variable
```

Objects

```
const user = { name: "Ana", age: 29, city: "Lisbon" };  
  
const { name, age } = user;           // name = "Ana", age = 29  
const { name: n, city: hometown = "Unknown" } = user; // rename + default  
  
function printUser({ name, age }) {   // destructure directly in params  
  console.log(`${name} (${age})`);  
}
```

Nested

```
const response = { data: { id: 1, tags: ["a", "b"] } };  
const { data: { id, tags: [firstTag] } } = response;  
// id = 1, firstTag = "a"
```

Enhanced object literals

Shorthand syntax for the most common patterns when building object literals.

Property shorthand

```
const name = "Ana", age = 29;

// before
const user = { name: name, age: age };

// ES6
const user2 = { name, age };
```

Method shorthand

```
const calculator = {
  // before: value: function() {...}
  add(a, b) {
    return a + b;
  }
};
```

Computed property names

```
const key = "role";
const user = {
  name: "Ana",
  [key]: "admin", // { name: "Ana", role: "admin" }
  [`is${key.charAt(0).toUpperCase()}${key.slice(1)}`]: true
};
```

for...of & iterables

A loop that walks any iterable — arrays, strings, Maps, Sets — yielding values directly, unlike `for...in` which yields keys.

```
const colors = ["red", "green", "blue"];

for (const color of colors) {
  console.log(color);
}

for (const char of "hi") { console.log(char); } // "h", "i"

const scores = new Map([["Ana", 95], ["Leo", 88]]);
for (const [name, score] of scores) {
  console.log(name, score);
}
```

FOR...IN

```
for (const i in colors) {
  console.log(i); // "0", "1",
  "2"
} // indices,
as strings
```

FOR...OF

```
for (const c of colors) {
  console.log(c); // "red",
  "green", "blue"
} // values
```

Use `for...of` for values; reserve `for...in` for enumerating an object's own and inherited keys.

Classes

Syntactic sugar over JavaScript's existing prototype-based inheritance — the same mechanism, a much more familiar shape.

```
class Animal {
  constructor(name) {
    this.name = name;
  }
  speak() {
    return `${this.name} makes a sound.`;
  }
  static create(name) {           // static: called on the class itself
    return new Animal(name);
  }
}

class Dog extends Animal {       // inheritance
  speak() {
    return `${super.speak()} It's a bark!`; // call the parent method
  }
}

const rex = new Dog("Rex");
rex.speak(); // "Rex makes a sound. It's a bark!"
```

Getters & setters

```
class Circle {
  constructor(radius) { this._radius = radius; }
  get area() { return Math.PI * this._radius ** 2; }
  set radius(value) {
    if (value < 0) throw new Error("Radius can't be negative");
    this._radius = value;
  }
}

new Circle(2).area; // accessed like a property, not called
```

Under the hood: a class is still a function, and instance methods still live on `ClassName.prototype`. Class bodies always run in strict mode, and a class declaration is not hoisted the way a function declaration is.

Modules

A native way to split code across files, replacing library-level solutions like CommonJS or AMD in the browser and, increasingly, in Node.js.

math.js exporting

```
export const PI = 3.14159;

export function add(a, b) {
  return a + b;
}

export default class Calculator {
  // only one default export per file
}
```

app.js importing

```
import Calculator, { PI, add } from
"./math.js";
import * as math from "./math.js";
// namespace import
import { add as sum } from "./math.js";
// renaming
```

In the browser, load a module with `<script type="module" src="app.js"></script>`. Module code runs in strict mode and each module has its own scope — nothing leaks to the global object by default.

Promises

An object representing the eventual result of an asynchronous operation, in one of three states: pending, fulfilled, or rejected.

```
function fetchUser(id) {
  return new Promise((resolve, reject) => {
    setTimeout(() => {
      if (id > 0) resolve({ id, name: "Ana" });
      else reject(new Error("Invalid id"));
    }, 500);
  });
}

fetchUser(1)
  .then(user => console.log(user))
  .catch(err => console.error(err))
  .finally(() => console.log("Done"));
```

Chaining & combinators

```
fetchUser(1)
  .then(user => fetchPosts(user.id))  // return a promise to chain it
  .then(posts => console.log(posts));

Promise.all([fetchUser(1), fetchUser(2)]);  // waits for every promise
Promise.race([fetchUser(1), timeout(1000)]);  // resolves with the first
```

Promises are the foundation `async/await` (ES2017) is built on — understanding them first makes that syntax much clearer.

Map & Set

Two new collection types that fix long-standing limitations of plain objects and arrays.

Map key → value, any key type

```
const scores = new Map();
scores.set("Ana", 95);
scores.set({ id: 1 }, "object key works too");

scores.get("Ana");           // 95
scores.has("Leo");           // false
scores.size;                 // 2
scores.delete("Ana");
```

Set unique values

```
const ids = new Set([1, 2, 2, 3]);
ids.size;                     // 3 –
                             duplicates dropped
ids.add(4);
ids.has(2);                   // true

const unique = [...new Set([1, 1, 2, 3])];
// [1, 2, 3] – a common dedupe idiom
```

Unlike plain objects, a Map's keys can be any type and it preserves insertion order; both **Map** and **Set** are directly iterable with `for...of`.

Symbols

A new primitive type whose values are always unique, used mainly to create property keys that won't collide with anything else.

```
const id = Symbol("id");
const user = {
  name: "Ana",
  [id]: 12345,           // hidden from normal enumeration
};

Object.keys(user);       // ["name"] – the symbol key is skipped
user[id];                // 12345 – still accessible directly

Symbol("id") === Symbol("id"); // false – every symbol is unique
```

Well-known symbols like `Symbol.iterator` are how ES6 lets an object opt into being iterable with `for...of` and the spread operator.

Generators

Functions that can pause mid-execution with `yield` and resume later, declared with an asterisk: `function*`.

```
function* countUp(max) {  
  for (let i = 1; i <= max; i++) {  
    yield i;  
  }  
}  
  
const counter = countUp(3);  
counter.next(); // { value: 1, done: false }  
counter.next(); // { value: 2, done: false }  
counter.next(); // { value: 3, done: false }  
counter.next(); // { value: undefined, done: true }  
  
for (const n of countUp(3)) console.log(n); // generators are iterable
```

Every generator function returns an iterator automatically — this is what powers custom `Symbol.iterator` implementations and, later, async generators.

Classes in depth

Beyond the ES6 basics: the class features added in later versions that are used in nearly every modern codebase, plus the patterns built on top of them.

Public & private fields

```
class BankAccount {
  balance = 0;           // public field, set per instance
  #pin;                  // private field: only accessible inside the class

  constructor(pin) {
    this.#pin = pin;
  }

  #verify(pin) {         // private method
    return pin === this.#pin;
  }

  withdraw(amount, pin) {
    if (!this.#verify(pin)) throw new Error("Wrong PIN");
    this.balance -= amount;
  }
}

const acc = new BankAccount(1234);
acc.#pin; // SyntaxError – private fields aren't accessible from outside
```

Static members & static blocks

```
class Config {
  static #instance;      // private static field
  static defaultTimeout = 3000; // public static field

  static {               // static initialization block (runs once)
    Config.#instance = new Config();
  }

  static get instance() { // classic singleton pattern
    return Config.#instance;
  }
}
```

Abstract-style base classes

```
class Shape {
  constructor() {
    if (new.target === Shape) {           // prevent direct instantiation
      throw new TypeError("Shape is abstract");
    }
  }
  area() {
    throw new Error("area() must be implemented by subclass");
  }
}

class Square extends Shape {
  constructor(side) { super(); this.side = side; }
  area() { return this.side ** 2; }
}
```

Mixins

JavaScript classes only support single inheritance. Mixins compose behavior from multiple sources using functions that return a class.

```
const Serializable = Base => class extends Base {
  toJSON() { return JSON.stringify(this); }
};
const Comparable = Base => class extends Base {
  equals(other) { return JSON.stringify(this) === JSON.stringify(other); }
};

class Point extends Comparable(Serializable(Object)) {
  constructor(x, y) { super(); this.x = x; this.y = y; }
}
```

instanceof & composition vs. inheritance

```
new Square(2) instanceof Shape;  // true – walks the prototype chain
```

A common guideline: prefer composition (objects holding other objects, or mixins) over deep inheritance chains. Two or three levels of `extends` is usually a sign to reach for composition instead.

JavaScript core

Everyday JavaScript features that landed after ES6, plus fundamentals worth having on one page alongside it.

async / await ES2017

```
async function getUser(id) {
  const res = await fetch(`/api/users/${id}`); // pauses until the promise settles
  if (!res.ok) throw new Error("Request failed");
  return res.json();
}

async function main() {
  try {
    const user = await getUser(1);
    console.log(user);
  } catch (err) {
    console.error(err);
  }
}
```

Syntactic sugar over promises (see Section 11): `await` unwraps a promise's resolved value, and a `try/catch` around it catches a rejection the way it would a thrown error.

Optional chaining & nullish coalescing ES2020

```
const city = user?.address?.city; // undefined instead of throwing, if address
is null
const call = obj?.method?.(); // safely calls a method that might not
exist

const name = user.name ?? "Anonymous"; // only falls back on null/undefined
const count = 0 ?? 10; // 0 – unlike ||, 0 is not replaced
```

Array & object methods worth knowing

METHOD	DOES
<code>arr.map(fn) / filter(fn) / reduce(fn, init)</code>	Transform, select, or fold an array
<code>arr.find(fn) / findIndex(fn)</code>	First match, or its index
<code>arr.includes(x) / flat() / flatMap(fn)</code>	Membership check; flatten nested arrays
<code>arr.at(-1)</code>	Negative indexing (last element)
<code>Object.entries(obj) / fromEntries(pairs)</code>	Object ↔ array of [key, value] pairs
<code>Object.freeze(obj)</code>	Prevent further changes to an object
<code>structuredClone(obj)</code>	Deep-copy an object without libraries

Equality & type coercion

```
0 == "0";           // true - == coerces types before comparing
0 === "0";          // false - === compares type and value
null == undefined;  // true (special case)
NaN === NaN;        // false - use Number.isNaN(x) instead
```

Rule of thumb: always use `===` / `!==`. Reach for `==` only in the rare case you deliberately want `null` and `undefined` to be treated the same.

TypeScript

A superset of JavaScript that adds a static type system, checked at compile time and erased before the code runs. Any valid JavaScript is already valid TypeScript.

Basic types

```
let id: number = 1;
let name: string = "Ana";
let active: boolean = true;
let tags: string[] = ["a", "b"];
let coords: [number, number] = [10, 20]; // tuple: fixed length & types
let status: "idle" | "loading" | "done" = "idle"; // union of literal types
let anything: unknown; // safer alternative to 'any'
```

Interfaces & type aliases

```
interface User {
  id: number;
  name: string;
  email?: string; // optional property
  readonly createdAt: Date; // can't be reassigned after creation
}

type ID = string | number; // type alias: also works for unions, primitives, etc.

function greet(user: User): string {
  return `Hi, ${user.name}`;
}
```

Functions

```
function add(a: number, b: number): number {
  return a + b;
}

const multiply = (a: number, b: number): number => a * b;

function log(message: string, level?: "info" | "error"): void {} // optional param
```

Generics

```
function firstOf<T>(items: T[]): T | undefined {
  return items[0];
}
firstOf<number>([1, 2, 3]);    // inferred as number
firstOf(["a", "b"]);           // T inferred automatically as string

interface Box<T> {
  value: T;
}
const stringBox: Box<string> = { value: "hi" };
```

Classes with types

```
class Animal {
  constructor(
    public name: string,          // parameter properties: declares + assigns in one
    step
    private age: number,
    protected readonly species: string = "unknown"
  ) {}

  speak(): string {
    return `${this.name} makes a sound.`;
  }
}

interface Speaker {
  speak(): string;
}
class Dog extends Animal implements Speaker {} // implements checks the shape
```

Narrowing

```
function printId(id: string | number) {
  if (typeof id === "string") {
    console.log(id.toUpperCase()); // TS knows id is a string here
  } else {
    console.log(id.toFixed(2));    // and a number here
  }
}
```

The compiler (tsc) checks types and then emits plain JavaScript — types add zero runtime cost. Configuration lives in a project's `tsconfig.json`, and `strict: true` is the recommended starting point for new projects.

Node.js

A JavaScript runtime built on Chrome's V8 engine that runs outside the browser — used for servers, CLI tools, and scripting, with access to the file system, network, and OS that browser JavaScript doesn't have.

Modules: CommonJS vs. ES Modules

COMMONJS .JS, DEFAULT

```
const fs = require("fs");

function add(a, b) { return a + b; }
module.exports = { add };
```

ES MODULES .MJS OR "TYPE": "MODULE"

```
import fs from "node:fs";

export function add(a, b) { return a + b; }
```

Reading files

```
import { readFile } from "node:fs/promises";

const data = await readFile("data.json", "utf-8");
const users = JSON.parse(data);

// synchronous and callback-based versions also exist:
// fs.readFileSync(path) / fs.readFile(path, callback)
```

A minimal HTTP server

```
import { createServer } from "node:http";

const server = createServer((req, res) => {
  res.writeHead(200, { "Content-Type": "application/json" });
  res.end(JSON.stringify({ message: "Hello" }));
});

server.listen(3000, () => console.log("Listening on :3000"));
```

Most real projects use a framework on top of this — Express or Fastify — for routing, middleware, and request parsing.

npm & package.json

```
{
  "name": "my-app",
  "version": "1.0.0",
  "type": "module",
  "scripts": { "start": "node index.js", "test": "node --test" },
  "dependencies": { "express": "^4.19.0" }
}
```

COMMAND	DOES
<code>npm install</code>	Installs everything in package.json
<code>npm install express</code>	Adds a dependency and saves it
<code>npm run <script></code>	Runs a script defined in package.json
<code>npm <tool></code>	Runs a package without installing it globally

The event loop & non-blocking I/O

```
console.log("1: start");

setTimeout(() => console.log("2: timer"), 0);
readFile("file.txt", () => console.log("3: file read"));

console.log("4: end");
// order: 1, 4, 2 or 3, 3 or 2 – sync code always runs first
```

Node is single-threaded for JavaScript execution but delegates I/O (file, network, timers) to the system, then processes the results via the event loop — this is why one Node process can handle many concurrent connections without spawning threads per request.

Environment & process

```
process.env.NODE_ENV;      // read environment variables
process.argv;               // command-line arguments
process.exit(1);            // exit with a status code
import.meta.url;            // current module's URL (ES module equivalent of
__filename)
```

Quick reference

Every feature on one page.

FEATURE	SYNTAX	USE IT FOR
<code>let / const</code>	<code>let x = 1; const y = 2;</code>	Block-scoped variables
Arrow function	<code>(a, b) => a + b</code>	Short callbacks, lexical <code>this</code>
Template literal	<code>`Hi \${name}`</code>	String interpolation, multi-line
Default parameter	<code>function f(x = 1) {}</code>	Optional arguments
Rest parameter	<code>function f(...args) {}</code>	Variable-length arguments
Spread	<code>[...arr], {...obj}</code>	Copying, merging, expanding
Array destructuring	<code>const [a, b] = arr</code>	Unpacking positional values
Object destructuring	<code>const {a, b} = obj</code>	Unpacking named properties
Property shorthand	<code>{ name }</code>	Building objects from variables
<code>for...of</code>	<code>for (const x of iterable)</code>	Iterating values
Class	<code>class X extends Y {}</code>	Prototype-based OOP, readably
Modules	<code>import / export</code>	Splitting code across files
Promise	<code>new Promise((res, rej) => {})</code>	Managing async results
<code>Map</code>	<code>new Map()</code>	Key/value pairs, any key type
<code>Set</code>	<code>new Set()</code>	Unique value collections
Symbol	<code>Symbol("desc")</code>	Unique, collision-free keys
Generator	<code>function* g() { yield x; }</code>	Pausable / lazy iteration
Private field	<code>#field</code>	Encapsulation inside a class
Static block	<code>static { ... }</code>	One-time class-level setup
<code>async / await</code>	<code>async function f() { await p; }</code>	Reading async code top-to-bottom
Optional chaining	<code>obj?.prop</code>	Safe access on possibly-null values
Nullish coalescing	<code>a ?? b</code>	Default only for null/undefined
TS interface	<code>interface X { a: string }</code>	Describing an object's shape
TS generic	<code>function f<T>(x: T): T</code>	Reusable, type-safe code

Node module	<code>import fs from "node:fs"</code>	Using Node's built-in modules
Node server	<code>createServer((req, res) => {})</code>	Handling HTTP requests

Compatibility note: ES6 (ES2015) is supported natively in every modern browser and in Node.js since v6. TypeScript compiles down to plain JavaScript, so it runs anywhere JavaScript does. Node.js ships its own release cadence separate from browser JavaScript versions — check a feature's Node compatibility separately when it matters.